ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)
(PermanentAffiliationby Andhra University&Approved by AICTE
Accredited by NBA (ECE, EEE, CSE, IT, Mech, Civil & Chemical) & NAAC with A+ Grade)
Sangivalasa-531162, Bheemunipatnam Mandal,
Visakhapatnam District

A PROJECT WORK REPORT ON..

SMART INVENTORY AND WAREHOUSE MANAGEMENT SYSTEM

This project is submitted by

k.SARANYA	A24125610287
A.LEKHANA PRIYA	A24126510267
E.TULASI RAM	A24126510277
M.BHARGAV NAIDU	A24126510297

UndertheGuidance of
DrB.MAHESH
Department of ComputerScience and Engineering
in partial fulfillment of therequirements for the award of the
offered by



B.TECH
IN
COMPUTER SCIENCEAND ENGINEERING
2025-2029

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)
(PermanentAffiliationby Andhra University&Approved by AICTE
Accredited by NBA (ECE, EEE, CSE, IT, Mech, Civil & Chemical) & NAAC with A+ Grade)
Sangivalasa-531162, Bheemunipatnam Mandal,
Visakhapatnam District



CERTIFICATE

This is to certify that this project work entitled —**"SMART INVENTORY AND WAREHOUSE MANAGEMENT SYSTEM"** is thebonafide record workdone by

k.SARANYA	A24125610287
A.LEKHANA PRIYA	A24126510267
E.TULASI RAM	A24126510277
M.BHARGAV NAIDU	A24126510297

of the Second Year,along with theirbatchmates, submitted in partial fulfillment of the requirement for the award of Bachelor of Technology (B.Tech) in Computer Science and Engineering to the Anil Neerukonda Institute of Technology and Sciences.

The results embodied in this project report have not been submitted to any other Board, University, or Institute for the award of any diploma or degree.

HEAD OF THE DEPARTMENT

LECTURER IN CHARGE

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us in successfully completing the project "SMART INVENTORY AND WAREHOUSE MANAGEMENT SYSTEM."

We are deeply thankful to our project guide Dr B. MAHESH sir, for providing us with valuable guidance, suggestions, and continuous support throughout the development of this project.

We would also like to express our gratitude to the Head of the Department and all the faculty members of the Department of Computer Science and Engineering for providing us with the necessary knowledge and encouragement to complete this project.

We sincerely thank our friends and classmates for their cooperation and support during the development and testing of the project.

Finally, we express our heartfelt thanks to our family members for their constant encouragement and support throughout the completion of this project.

This project has helped us gain practical knowledge of C programming, structures, arrays, searching, sorting, and queue data structures, and understand their application in a real-world inventory management system.

ABSTRACT

The Smart Inventory and Warehouse Management System is a C-based application designed to simplify and organize basic inventory and warehouse operations. The system helps manage product details, supplier information, stock quantities, and customer orders in a structured manner.

The system provides different operations such as adding and displaying products, adding and displaying suppliers, updating stock, displaying inventory, searching for products, sorting products based on price, adding and processing orders, displaying orders, and generating low-stock alerts.

The project uses fundamental concepts of C programming and data structures, including structures, arrays, linear searching, Bubble Sort, and queue operations. The order management module follows the FIFO (First-In-First-Out) principle, where the earliest order entered into the queue is processed first.

The main objective of this project is to provide a simple and efficient computerized solution for managing inventory and warehouse-related activities. It also demonstrates the practical application of basic programming and data structure concepts to solve a real-world problem.

Overall, the project provides a clear understanding of how C programming, searching, sorting, and queue data structures can be combined to develop a basic inventory management system.

PREFACE

Table of contents

Sl.No.	Chapter Title	Sub-Contents	23-28
1	Introduction	• Overview of the Project • Purpose and Scope • Existing System and Proposed System	1
2	Problem Definition	• Problem Statement • Objectives of the System	2
3	System Design	• System Architecture • Data Structures Used • Algorithms Used • Flowcharts • Data Flow Diagrams • Visual Layout	3-10
4	Implementation Details	• Program Structure • Language and Environment Used • Key Functional Modules • Code Snippets • File Handling Explanation	11-13
5	Results and Discussion	• Output Screens • Test Case Table (Input, Expected Output, Actual Output)	14-20
6	Conclusion	• Summary of Achievements • Limitations of the System • Future Enhancements	21
7	References	• Books Referred • Classroom Notes and Reference Materials	22
8	Appendix	• Source Code Listing	23-30

1.INTRODUCTION

1.1 Overview of the Project

The Smart Inventory and Warehouse Management System is a menu-driven application developed using the C programming language. The system is designed to manage basic warehouse activities such as products, suppliers, inventory stock, and customer orders in an organized manner.

The application provides different options through a main menu, allowing the user to add and display products, manage suppliers, update stock, search for products, sort products based on price, manage orders, and identify products with low stock.

1.2 Purpose and Scope

The main purpose of the project is to provide a simple computerized system for managing inventory-related activities. It reduces the effort required to maintain and check product and stock information manually.

The scope of the system includes product management, supplier management, inventory management, product searching, price-based sorting, order processing, and low-stock monitoring.

1.3 Existing System and Proposed System

Existing System

In a manual inventory system, product details, stock quantities, supplier information, and orders may be maintained manually. Searching for products, updating stock, and identifying low-stock items can require additional time and effort.

Proposed System

The proposed system provides a simple menu-driven solution for performing these operations using C programming. It uses structures, arrays, searching, sorting, and queue operations to organize and process inventory and order information efficiently.

2. PROBLEM DEFINITION

Managing inventory manually can make it difficult to maintain product details, supplier information, stock quantities, and customer orders in an organized manner. Searching for a particular product, updating stock, checking product prices, processing orders, and identifying low-stock products can require additional time and effort.

Therefore, there is a need for a simple computerized system that can organize these basic warehouse operations and provide the required information through an easy-to-use menu-driven interface.

2.2 Objectives of the System

The main objectives of the Smart Inventory and Warehouse Management System are:

1. To maintain product details in an organized manner.
2. To store and display supplier information.
3. To update and display available stock quantities.
4. To search for products using their product ID.
5. To sort products according to their price.
6. To manage customer orders using a FIFO queue.
7. To identify products whose stock reaches the reorder level.
8. To demonstrate the practical use of C programming, structures, arrays, searching, sorting, and queue operations.

3. SYSTEM DESIGN

3.1 System Architecture

The Smart Inventory and Warehouse Management System follows a simple menu-driven architecture. The user selects an operation from the main menu, and the corresponding function performs the required task on the stored product, supplier, inventory, or order data.

User → Main Menu → Selected Function → Data Processing → Output

The major functional components of the system are:

- Product Management
- Supplier Management
- Inventory Management
- Product Search
- Product Sorting
- Order Management
- Low Stock Alert

3.2 Data Structures Used

The system uses structures and arrays to store and manage records.

Product Structure

Stores product ID, product name, supplier ID, quantity, price, and reorder level.

Supplier Structure

Stores supplier ID, supplier name, and contact information.

Order Structure

Stores order ID, product ID, and order quantity.

Arrays

Arrays are used to maintain multiple product, supplier, and order records.

Queue

A queue is used for order management. Orders follow the FIFO (First-In-First-Out) principle, where the first order added is processed first.

3.3 Algorithms Used

- Linear Search: Used to search for a product using its product ID.
- Bubble Sort: Used to sort products according to their price.
- Queue Operations: Used to add and process customer orders.
- Stock Update: Used to increase the quantity of an existing product.
- Low Stock Check: Compares product quantity with its reorder level and displays products requiring attention.

3.3 ALGORITHMS USED — CONTINUATION

1. Product Addition Algorithm

- Check whether the product storage is full.
- Enter product ID, name, supplier ID, quantity, price, and reorder level.
- Store the details in the product array.
- Increase the product count.
- Display a successful addition message.

2. Product Display Algorithm

- Check whether products are available.
- Traverse the product array.
- Display the details of each product.
- Continue until all products are displayed.

3. Supplier Addition Algorithm

- Enter supplier ID, supplier name, and contact number.
- Store the supplier details in the supplier array.
- Increase the supplier count.
- Display a successful addition message.

4. Stock Update Algorithm

- Enter the product ID.
- Search for the product in the product array.
- If the product is found, display its current quantity.
- Enter the quantity to be added.
- Update the stock quantity.
- Display the updated quantity.

5. Product Search Algorithm

- Enter the product ID.
- Search the product array sequentially.
- Compare each product ID with the entered ID.
- Display the product details if a match is found.
- Otherwise, display “Product Not Found”.

6. Product Sorting Algorithm

- Compare the prices of adjacent products.
- Swap the products if they are in the wrong order.
- Repeat the process for all products.
- Display the products in ascending order of price.

7. Low Stock Alert Algorithm

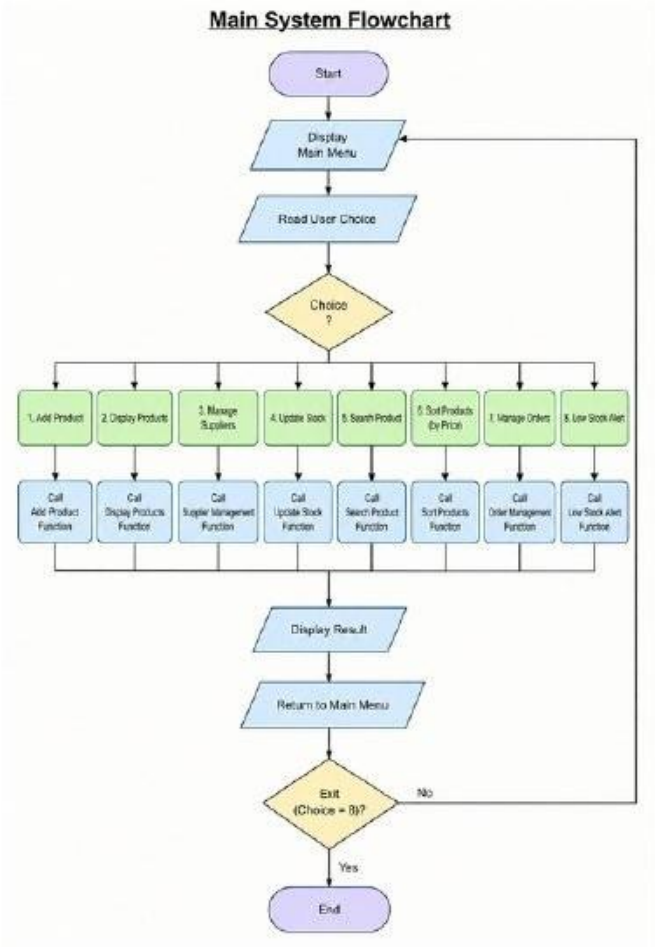
- Traverse all product records.
- Compare the quantity with the reorder level.
- If quantity is less than or equal to the reorder level, display the product.
- Continue until all products are checked.

8. **Order Queue Algorithm**

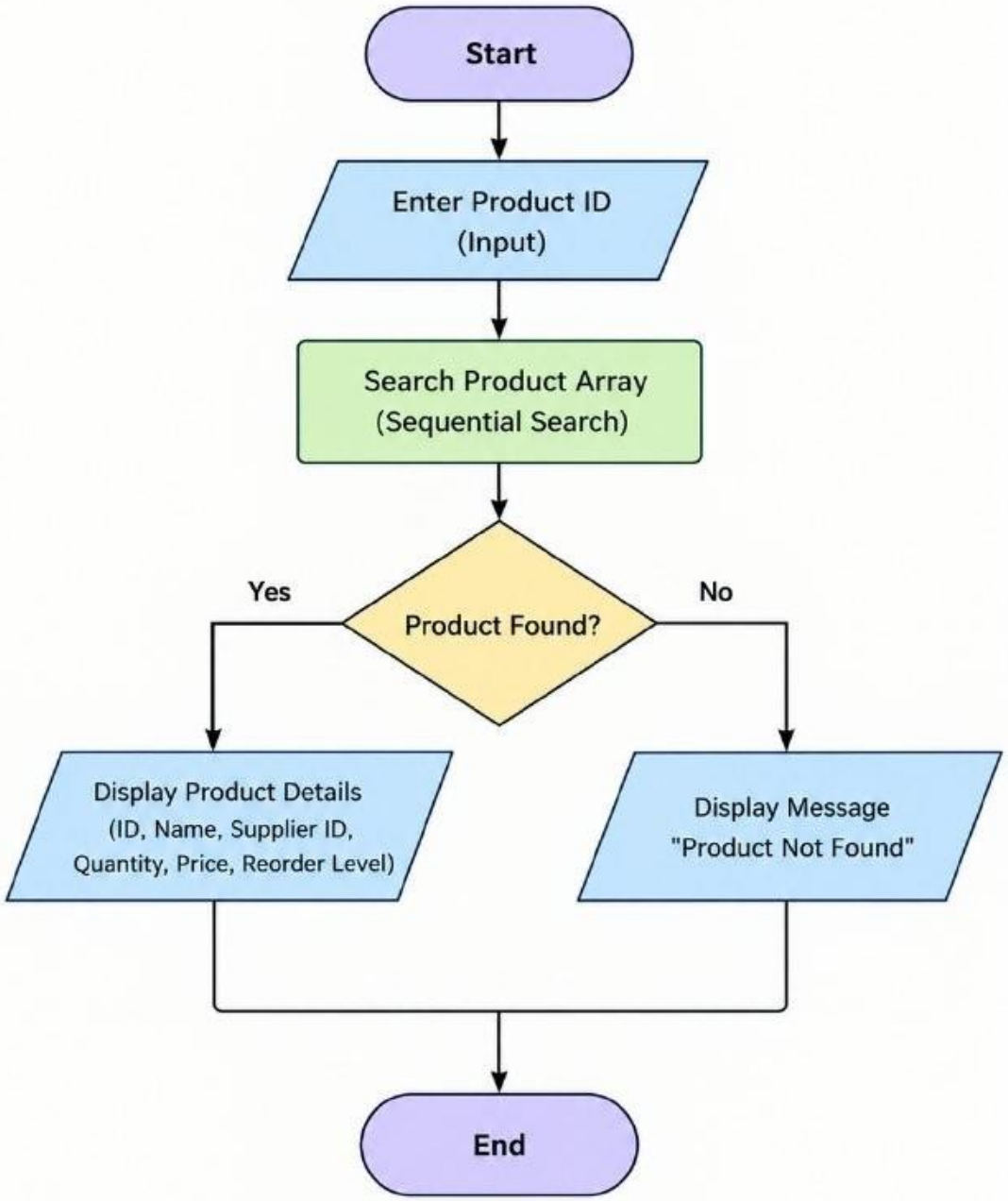
- Accept the order ID, product ID, and required quantity.
- Check whether the queue is full.
- Add the order at the rear of the queue.
- If the queue is empty, set the front position.
- Display a confirmation message after adding the order.

9. Order Processing Algorithm

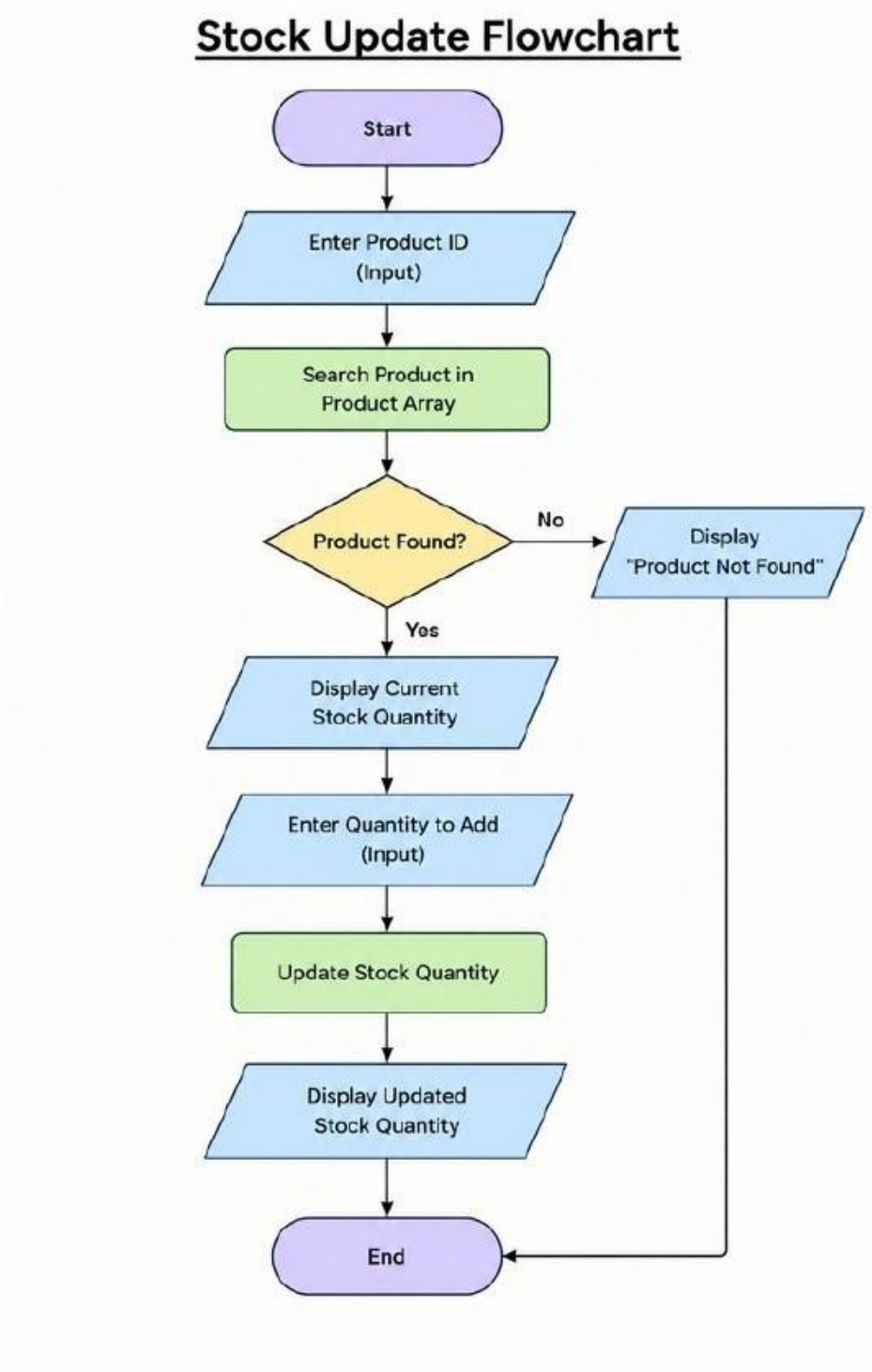
- Check whether the order queue is empty.
- Retrieve the order from the front of the queue.
- Find the corresponding product.
- Check whether sufficient stock is available.
- If sufficient stock exists, reduce the product quantity.
- Move the front position to the next order.
- Display the order processing result.



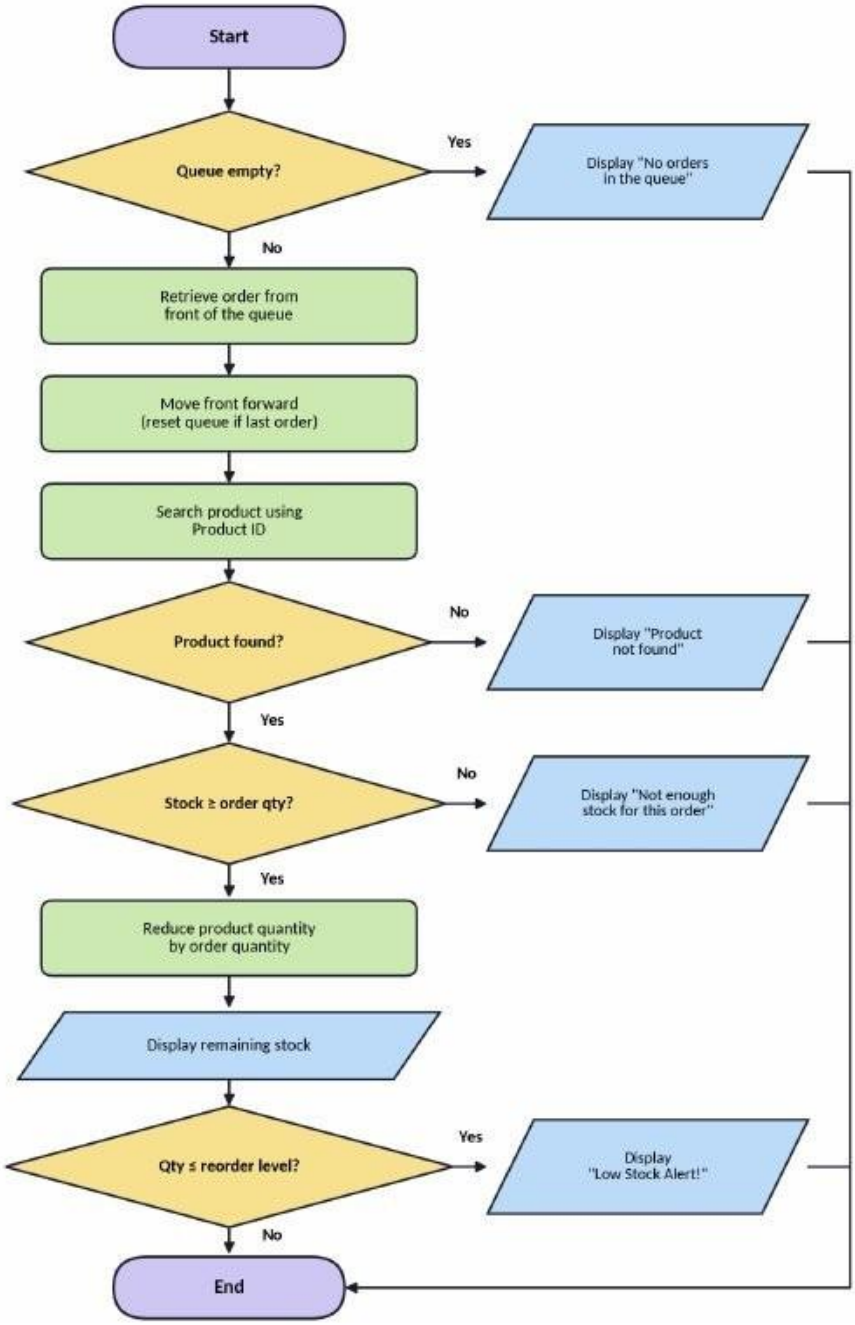
Product Search Flowchart



Stock Update Flowchart



Order Processing Flowchart



Data flow:

Level 0 DFD (Context Diagram)

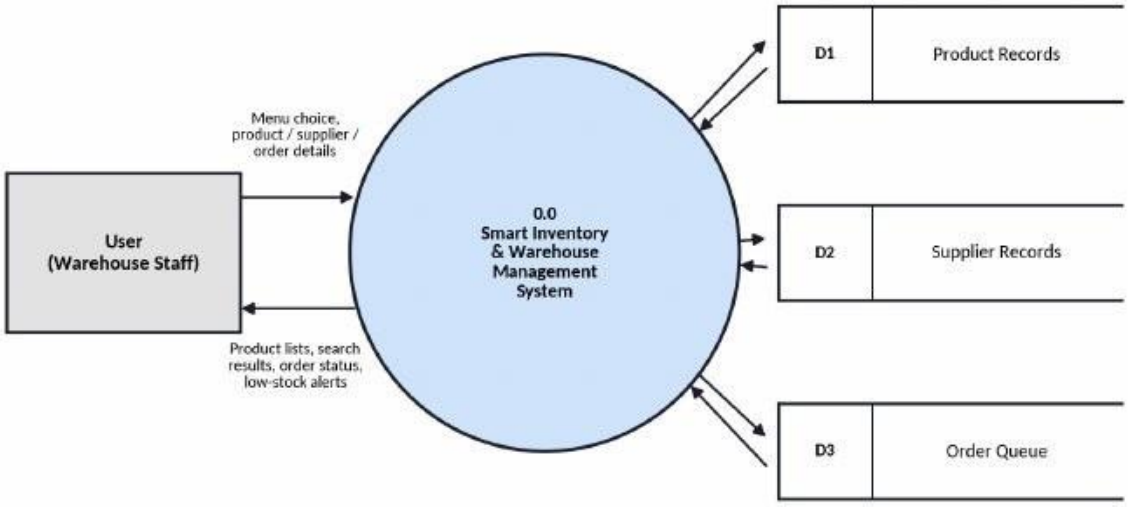


Fig 3.1 Level 0 Data Flow Diagram of the Smart Inventory and Warehouse Management System

Level 1 DFD

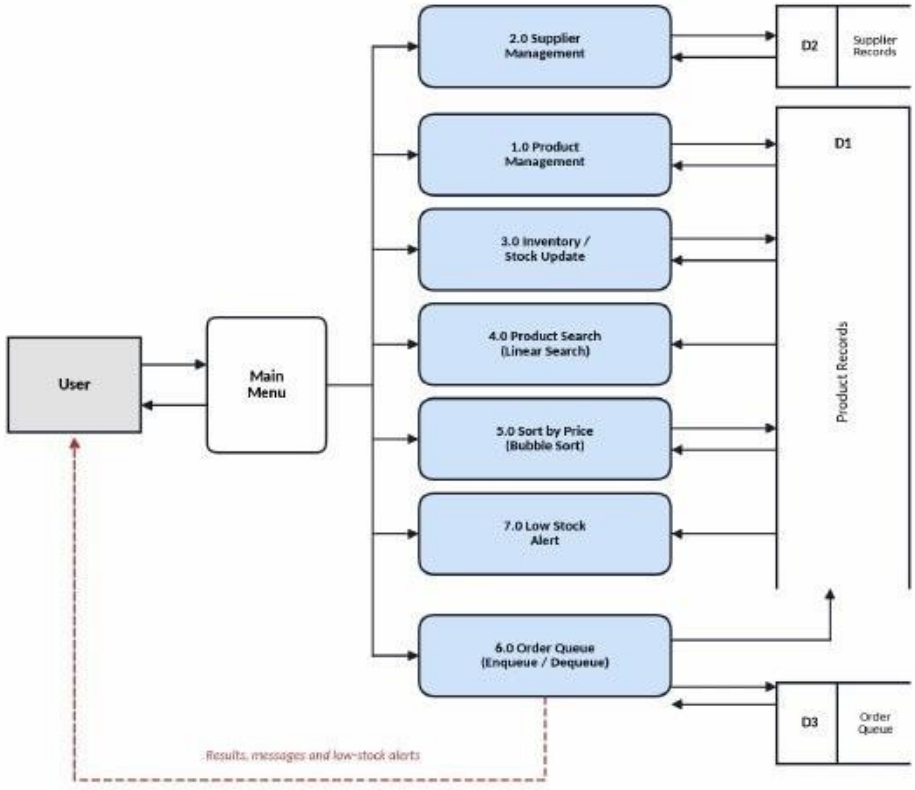


Fig 3.2 Level 1 Data Flow Diagram showing the seven modules and the three data stores

Visual layout:

The system uses a console (text-based) interface. When the program starts, the main menu shown below is displayed. The user selects an operation by typing its number, and the program returns to this menu after every operation until option 13 (Exit) is chosen.

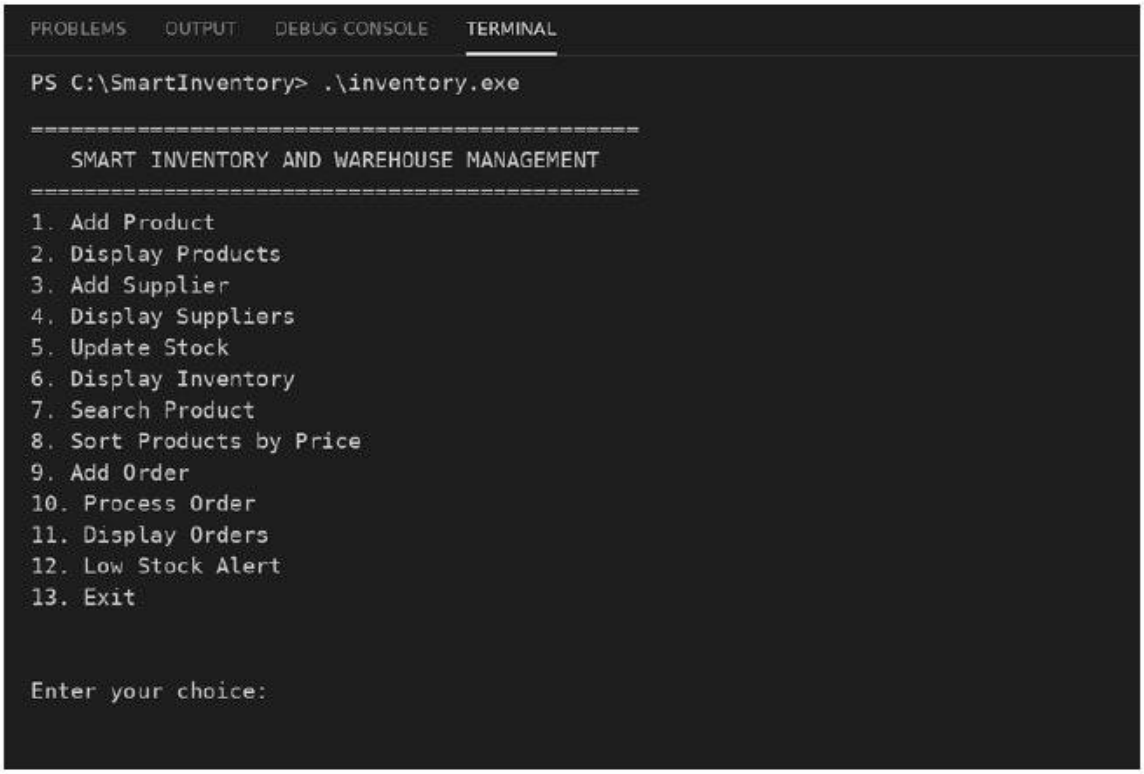


Fig 3.3 Main menu of the application (Visual Studio Code terminal)

Menu options and the modules they use

Option	Operation	Module
1, 2	Add Product, Display Products	Product Management
3, 4	Add Supplier, Display Suppliers	Supplier Management
5, 6	Update Stock, Display Inventory	Inventory Management
7	Search Product	Searching (Linear Search)
8	Sort Products by Price	Sorting (Bubble Sort)
9, 10, 11	Add Order, Process Order, Display Orders	Order Queue (FIFO)
12	Low Stock Alert	Low Stock Alert
13	Exit	Terminates the program

4. IMPLEMENTATION

4.1 Program Structure

The Smart Inventory and Warehouse Management System is implemented using the C programming language. The program follows a menu-driven approach, where the user selects the required operation from the main menu.

The program uses separate functions for different operations such as adding products, displaying products, managing suppliers, updating stock, searching products, sorting products, managing orders, and checking low-stock products. The main function controls the execution of the program and calls the required function based on the user's choice.

The program is organised into the following parts:

- **Header files and constants:** stdio.h and string.h are included; MAX_PRODUCTS (100), MAX_SUPPLIERS (50) and MAX_ORDERS (5) define the storage limits.
- **Structures:** struct Product, struct Supplier and struct Order store the records.
- **Global arrays and counters:** products[], suppliers[] and the order queue q[] with productCount, supplierCount and the queue pointers f (front) and r (rear).
- **Functions:** one function for each menu option.
- **main():** displays the menu in an infinite loop and calls the selected function until Exit is chosen.

4.2 Language and Environment Used

- **Programming language:** C
- **Editor / IDE:** Visual Studio Code with a C compiler (GCC)
- **Header files:** stdio.h, string.h
- **Interface:** console (command-line) menu-driven interface
- **Operating system:** Windows

4.3 Key Functional Modules

The system consists of the following major modules:

- **Product Management:** Used to add and display product details.
- **Supplier Management:** Used to add and display supplier information.
- **Stock Management:** Used to update and display product stock.
- **Product Search:** Searches for a product using its product ID.
- **Product Sorting:** Sorts products according to their price.
- **Order Management:** Adds and processes orders using a FIFO queue.
- **Low Stock Alert:** Identifies products whose quantity is less than or equal to the reorder level.

4.4 Program Execution

When the program starts, the main menu is displayed on the screen. The user selects an option by entering the corresponding number. The program then calls the required function and performs the selected operation.

After completing the operation, the result is displayed to the user and the program returns to the main menu. This process continues until the user selects the Exit option.

4.5 Key Functional Modules and Code Snippets

Product Structure

```
struct Product
{
    int productId;
    char productName[50];
    int supplierId;
    int quantity;
    float price;
    int reorderLevel;
};
```

Supplier Structure

```
struct Supplier
{
    int supplierId;
    char supplierName[50];
    char contact[20];
};
```

Order Structure

```
struct Order
{
    int orderId;
    int productId;
    int quantity;
};
```

Linear Search (search by Product ID)

```
for (int i = 0; i < productCount; i++)
{
    if (products[i].productId == id)
    {
        printf("\nProduct found!\n");
        /* display product details */
        found = 1;
        break;
    }
}
if (!found)
    printf("\nProduct not found!\n");
```

Bubble Sort (sort by price)

```
for (i = 0; i < productCount - 1; i++)
{
    for (j = 0; j < productCount - i - 1; j++)
    {
        if (products[j].price > products[j + 1].price)
        {
            temp = products[j];
            products[j] = products[j + 1];
            products[j + 1] = temp;
        }
    }
}
```

Order Queue: Enqueue (add order at the rear)

```
if (r == MAX_ORDERS - 1)
{
    printf("\nQueue is full.\n");
    return;
}
...
if (f == -1)
    f = 0;

r++;
q[r] = newOrder;
```

Order Queue: Dequeue (process the first order)

```
if (f == -1)
{
    printf("\nNo orders in the queue.\n");
    return;
}

o = q[f];

if (f == r)
    f = r = -1;      /* queue becomes empty */
else
    f++;             /* move front to next order */

if (products[i].quantity >= o.quantity)
    products[i].quantity -= o.quantity;
else
    printf("Not enough stock for this order\n");
```

Low Stock Check

```
for (int i = 0; i < productCount; i++)
{
    if (products[i].quantity <= products[i].reorderLevel)
    {
        printf("\nLow Stock Alert!\n");
        /* display product details */
    }
}
```

4.6 File Handling Explanation

The current version of the project does not use file handling. All product, supplier and order records are stored in global arrays (products[], suppliers[] and q[]) in the computer's memory while the program is running. When the program is closed, the stored data is cleared and has to be entered again the next time the program is run.

This keeps the program simple and demonstrates the core ideas of structures, arrays, searching, sorting and queues. In a future version, C file-handling functions such as fopen(), fprintf(), fscanf() and fclose() can be used to save the product, supplier and order records in a file so that the data remains available after the program is closed.

5. RESULTS AND DISCUSSION

5.1 Output Screens

The following screens show the output of the program when it is run in the Visual Studio Code terminal. The sample data used is: Icecream (ID 103, Rs. 200), Soap (ID 101, Rs. 50) and Chocolate (ID 102, Rs. 70); suppliers ABC Traders and Fresh Foods Pvt Ltd.



Fig 5.1 Add Product (option 1)

The user enters the product ID, name, supplier ID, quantity, price and reorder level. The system confirms that the product has been added.



Fig 5.2 Display Products (option 2)

All stored products are displayed with their ID, name, supplier ID, quantity and price.

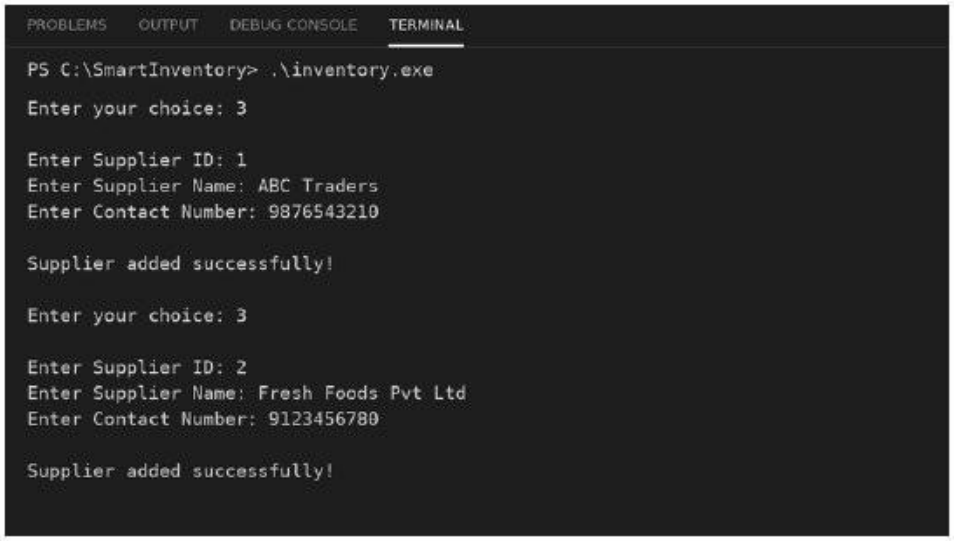


Fig 5.3 Add Supplier (option 3)

Supplier ID, name and contact number are stored in the supplier array.



Fig 5.4 Display Suppliers (option 4)

The stored suppliers are displayed in a table.

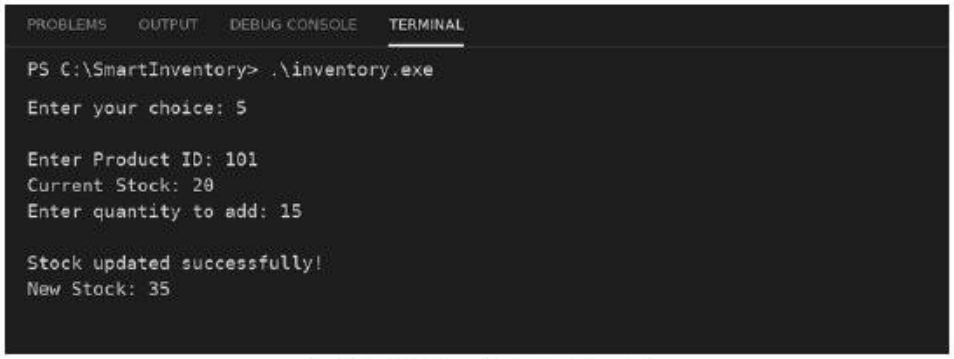


Fig 5.5 Update Stock (option 5)

The current stock is shown and the entered quantity is added. Soap stock changes from 20 to 35.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL

PS C:\SmartInventory> .\inventory.exe

Enter your choice: 6

ID     Name            Quantity   Reorder Level  Status
-------------------------------------------------------
103    Icecream        30         5              OK
101    Soap            35         10             OK
102    Chocolate       8          10             LOW
```

Fig 5.6 Display Inventory (option 6)

Shows quantity, reorder level and a stock status (OK / LOW) for each product.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL

PS C:\SmartInventory> .\inventory.exe

Enter your choice: 7

Enter Product ID: 102

Product found!

Product ID: 102
Product Name: Chocolate
Product Price: 70.00
Product Quantity: 8

Enter your choice: 7

Enter Product ID: 999

Product not found!
```

Fig 5.7 Search Product (option 7)

Linear Search finds product ID 102; searching for ID 999 shows "Product not found!".

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL

PS C:\SmartInventory> .\inventory.exe

Enter your choice: 8

Products sorted by price (low to high):

ID     Name            Supplier  Qty    Price
-------------------------------------------------
101    Soap            2         35     50.00
102    Chocolate       2         8      70.00
103    Icecream        1         30     200.00
```

Fig 5.8 Sort Products by Price (option 8)

Bubble Sort arranges the products in ascending order of price (Soap, Chocolate, Icecream).

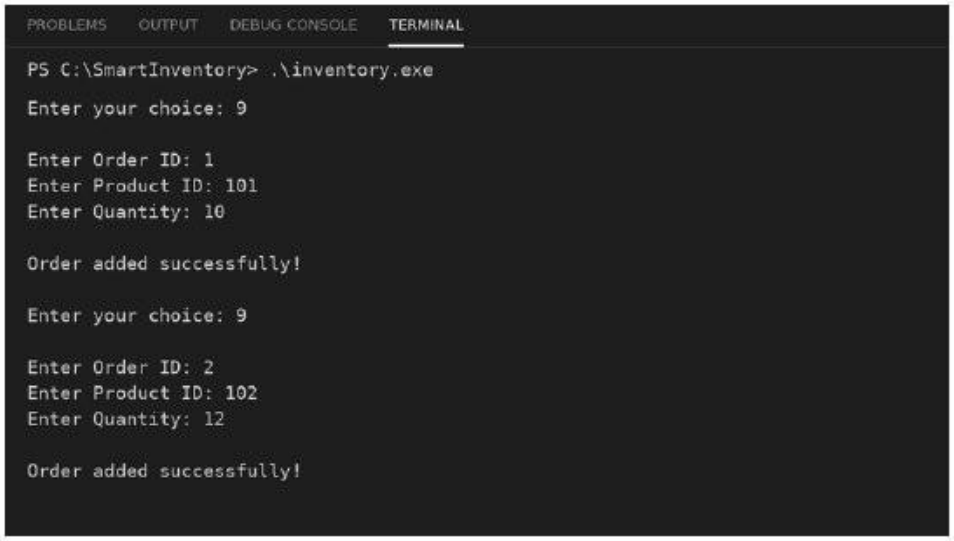


Fig 5.9 Add Order (option 9)

Orders are added at the rear of the queue.



Fig 5.10 Display Orders (option 11)

Pending orders are listed from front to rear (FIFO order).

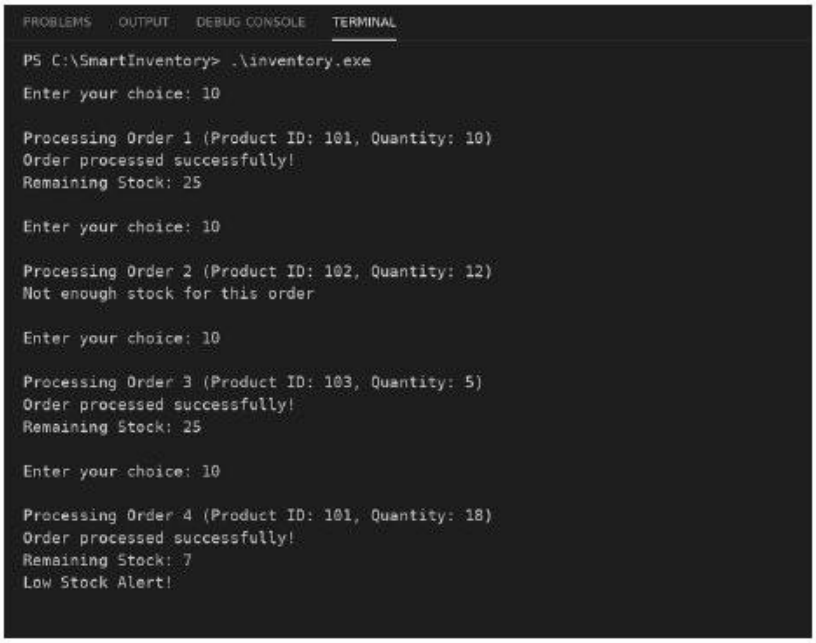


Fig 5.11 Process Order (option 10)

Order 1 is processed first. Order 2 fails because only 8 units of Chocolate are available. Order 4 reduces Soap to 7, which triggers a low-stock alert.



Fig 5.12 Low Stock Alert (option 12)

Products whose quantity is less than or equal to the reorder level are listed.



Fig 5.13 Invalid choice and Exit (option 13)

An invalid option shows an error message and the program returns to the menu. Option 13 ends the program.

5.2 Test Case Table

Each test was run on the compiled program. The expected output was written before the run and compared with the actual output.

TC No.	Test Case	Input	Expected Output	Actual Output	Result
1	Add a product	Option 1: ID 101, Soap, Supplier 1, Qty 20, Price 50, Reorder 10	Product added successfully!	Product added successfully!	**Pass**
2	Display products when none are stored	Option 2 (no products added)	No products available.	No products available.	**Pass**
3	Display products	Option 2 after adding Icecream, Soap, Chocolate	103 Icecream 101 Soap	ID Name Supplier Qty Price 103 Icecream 1 30 200.00 101 Soap 2 20 50.00 102 Chocolate 2 8 70.00	**Pass**
4	Add a supplier	Option 3: ID 1, ABC Traders, 9876543210	Supplier added successfully!	Supplier added successfully!	**Pass**
5	Update stock of an existing product	Option 5: ID 101, add 15 (current stock 20)	Current Stock: 20 New Stock: 35	Current Stock: 20 Stock updated successfully! New Stock: 35	**Pass**
6	Update stock of a missing product	Option 5: ID 999	Product not found!	Product not found!	**Pass**
7	Search an existing product	Option 7: ID 102	Product found! Product Name: Chocolate Product Price: 70.00 Product Quantity: 8	Product found! Product ID: 102 Product Name: Chocolate Product Price: 70.00 Product Quantity: 8	**Pass**
8	Search a missing product	Option 7: ID 999	Product not found!	Product not found!	**Pass**
9	Sort products by price	Option 8 (stored order: 200, 50, 70)	Soap Chocolate Icecream	Products sorted by price (low to high): ID Name Supplier Qty Price 101 Soap 2 20 50.00 102 Chocolate 2 8 70.00 103 Icecream 1 30 200.00	**Pass**
10	Add an order to the queue	Option 9: Order 1, Product 101, Qty 5	Order added successfully!	Order added successfully!	**Pass**
11	Process order (enough stock)	Option 10 with Order 1: Product 101, Qty 5 (stock 20)	Order processed successfully! Remaining Stock: 15	Processing Order 1 (Product ID: 101, Quantity: 5) Order processed successfully! Remaining Stock: 15	**Pass**
12	Process order (not enough stock)	Option 10 with Order 1: Product 101, Qty 50 (stock 20)	Not enough stock for this order	Processing Order 1 (Product ID: 101, Quantity: 50) Not enough stock for this order	**Pass**
13	Process order that reaches reorder level	Option 10 with Order 1: Product 101, Qty 12 (stock 20, reorder 10)	Remaining Stock: 8 Low Stock Alert!	Processing Order 1 (Product ID: 101, Quantity: 12) Order processed successfully! Remaining Stock: 8 Low Stock Alert!	**Pass**
14	Process order from an empty queue	Option 10 (no orders added)	No orders in the queue.	No orders in the queue.	**Pass**

TC No.	Test Case	Input	Expected Output	Actual Output	Result
15	FIFO order processing	Orders 1 and 2 added, Option 10 once	Processing Order 1 Remaining Stock: 18	Processing Order 1 (Product ID: 101, Quantity: 2) Order processed successfully! Remaining Stock: 18	**Pass**
16	Queue full	Option 9 six times (MAX_ORDERS = 5)	Queue is full.	Queue is full.	**Pass**
17	Low stock alert	Option 12 after adding Icecream, Soap, Chocolate	Low Stock Alert! Product Name: Chocolate Current Stock: 8 Reorder Level: 10	Low Stock Alert! Product ID: 102 Product Name: Chocolate Current Stock: 8 Reorder Level: 10	**Pass**
18	Invalid menu choice	Option 99	Invalid choice!	Invalid choice!	**Pass**

All 18 test cases produced the expected output.

6. CONCLUSION

6.1 Summary of Achievements

The Smart Inventory and Warehouse Management System successfully provides a simple and organized solution for managing basic warehouse activities. The system allows users to maintain product and supplier details, update stock, search products, sort products based on price, manage customer orders, and identify low-stock products.

The project demonstrates the practical implementation of important C programming and data structure concepts such as structures, arrays, functions, linear searching, Bubble Sort, and Queue operations. The order management module follows the FIFO (First-In-First-Out) principle for processing orders.

Overall, this project helped in understanding how programming concepts can be applied to solve a real-world inventory management problem.

6.2 Limitations of the System

- Data is stored only in memory, so all records are lost when the program is closed.
- The storage capacity is fixed (100 products, 50 suppliers and 5 orders at a time).
- The order queue is a linear array, so the space of processed orders can be reused only after the queue becomes empty.
- Duplicate product IDs and supplier IDs are not checked, and the input is not validated.
- Stock update can only add stock; stock is reduced only by processing an order.
- Each order contains only one product.
- Linear Search and Bubble Sort are simple but slow for very large amounts of data.
- The system has a console interface only and has no user login or authentication.

6.3 Future Enhancements

- Store records permanently using file handling or a database.
- Use a circular queue or linked list so that the order queue has no fixed small limit.
- Add user authentication for different roles such as administrator and staff.
- Add validation for duplicate IDs and invalid input.
- Add options to edit and delete products and suppliers, and to reduce or correct stock.
- Allow an order to contain more than one product and generate bills.
- Use faster algorithms such as Binary Search and Quick Sort for large data.
- Add advanced reports and a graphical user interface.

7. REFERENCES

1. Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language, Prentice Hall.

2. Yashavant Kanetkar, Let Us C, BPB Publications.

3. Reema Thareja, Data Structures Using C, Oxford University Press.

4. E. Balagurusamy, Programming in ANSI C, McGraw Hill Education.

5. Classroom Notes and Course Material related to C Programming and Data Structures.

6. C Programming Reference Materials used for understanding structures, arrays, functions, searching, sorting, and queue operations.

7. Data Structures Reference Materials used for understanding Linear Search, Bubble Sort, and FIFO Queue implementation.

8. APPENDIX

Source Code (inventory.c)

```
#include <stdio.h>
#include <string.h>

#define MAX_PRODUCTS 100
#define MAX_SUPPLIERS 50
#define MAX_ORDERS 5

struct Product
{
    int productId;
    char productName[50];
    int supplierId;
    int quantity;
    float price;
    int reorderLevel;
};

struct Supplier
{
    int supplierId;
    char supplierName[50];
    char contact[20];
};

struct Order
{
    int orderId;
    int productId;
    int quantity;
};

struct Product products[MAX_PRODUCTS];
struct Supplier suppliers[MAX_SUPPLIERS];
struct Order q[MAX_ORDERS];

int productCount = 0;
int supplierCount = 0;

int f = -1;
int r = -1;

void addProduct()
{
    if (productCount >= MAX_PRODUCTS)
    {
        printf("\nProduct storage is full!\n");
        return;
    }

    printf("\nEnter Product ID: ");
    scanf("%d", &products[productCount].productId);

    printf("Enter Product Name: ");
    scanf(" %[^\n]", products[productCount].productName);

    printf("Enter Supplier ID: ");
    scanf("%d", &products[productCount].supplierId);

    printf("Enter Quantity: ");
    scanf("%d", &products[productCount].quantity);

    printf("Enter Price: ");
    scanf("%f", &products[productCount].price);

    printf("Enter Reorder Level: ");
    scanf("%d", &products[productCount].reorderLevel);

    productCount++;

    printf("\nProduct added successfully!\n");
}

void displayProducts()
```

```
 {
     int i;

    if (productCount == 0)
    {
        printf("\nNo products available.\n");
        return;
    }

    printf("\n%-6s %-16s %-9s %-6s %s\n", "ID", "Name", "Supplier", "Qty", "Price");
    printf("-------------------------------------------------\n");
    for (i = 0; i < productCount; i++)
    {
        printf("%-6d %-16s %-9d %-6d %.2f\n",
               products[i].productId,
               products[i].productName,
               products[i].supplierId,
               products[i].quantity,
               products[i].price);
    }
}

void addSupplier()
{
    if (supplierCount >= MAX_SUPPLIERS)
    {
        printf("\nSupplier storage is full!\n");
        return;
    }

    printf("\nEnter Supplier ID: ");
    scanf("%d", &suppliers[supplierCount].supplierId);

    printf("Enter Supplier Name: ");
    scanf(" %[^\n]", suppliers[supplierCount].supplierName);

    printf("Enter Contact Number: ");
    scanf("%s", suppliers[supplierCount].contact);

    supplierCount++;

    printf("\nSupplier added successfully!\n");
}

void displaySuppliers()
{
    int i;

    if (supplierCount == 0)
    {
        printf("\nNo suppliers available.\n");
        return;
    }

    printf("\n%-6s %-24s %s\n", "ID", "Name", "Contact");
    printf("-------------------------------------------------\n");
    for (i = 0; i < supplierCount; i++)
    {
        printf("%-6d %-24s %s\n",
               suppliers[i].supplierId,
               suppliers[i].supplierName,
               suppliers[i].contact);
    }
}

void updateStock()
{
    int id, quantity, i, found = 0;

    printf("\nEnter Product ID: ");
    scanf("%d", &id);

    for (i = 0; i < productCount; i++)
    {
        if (products[i].productId == id)
        {
            printf("Current Stock: %d\n",
```

```
                products[i].quantity);

            printf("Enter quantity to add: ");
            scanf("%d", &quantity);

            products[i].quantity += quantity;

            printf("\nStock updated successfully!\n");
            printf("New Stock: %d\n",
                products[i].quantity);

            found = 1;
            break;
        }
    }

    if (!found)
        printf("\nProduct not found!\n");
}

void displayInventory()
{
    int i;

    if (productCount == 0)
    {
        printf("\nInventory is empty.\n");
        return;
    }

    printf("\n%-6s %-16s %-10s %-14s %s\n", "ID", "Name", "Quantity", "Reorder Level", "Status");
    printf("-------------------------------------------------------------\n");
    for (i = 0; i < productCount; i++)
    {
        printf("%-6d %-16s %-10d %-14d %s\n",
            products[i].productId,
            products[i].productName,
            products[i].quantity,
            products[i].reorderLevel,
            (products[i].quantity <= products[i].reorderLevel) ? "LOW" : "OK");
    }
}

void search()
{
    int id;
    int found = 0;

    printf("\nEnter Product ID: ");
    scanf("%d", &id);

    for (int i = 0; i < productCount; i++)
    {
        if (products[i].productId == id)
        {
            printf("\nProduct found!\n");
            printf("\nProduct ID: %d",
                products[i].productId);
            printf("\nProduct Name: %s",
                products[i].productName);
            printf("\nProduct Price: %.2f",
                products[i].price);
            printf("\nProduct Quantity: %d\n",
                products[i].quantity);
            found = 1;
            break;
        }
    }

    if (!found)
        printf("\nProduct not found!\n");
}

void sort()
{
```

```
    int i, j;
    struct Product temp;

    if (productCount == 0)
    {
        printf("\nNo products available.\n");
        return;
    }

    for (i = 0; i < productCount - 1; i++)
    {
        for (j = 0; j < productCount - i - 1; j++)
        {
            if (products[j].price > products[j + 1].price)
            {
                temp = products[j];
                products[j] = products[j + 1];
                products[j + 1] = temp;
            }
        }
    }

    printf("\nProducts sorted by price (low to high):\n");
    displayProducts();
}

void enq()
{
    if (r == MAX_ORDERS - 1)
    {
        printf("\nQueue is full.\n");
        return;
    }

    struct Order newOrder;

    printf("\nEnter Order ID: ");
    scanf("%d", &newOrder.orderId);

    printf("Enter Product ID: ");
    scanf("%d", &newOrder.productId);

    printf("Enter Quantity: ");
    scanf("%d", &newOrder.quantity);

    if (f == -1)
        f = 0;

    r++;
    q[r] = newOrder;

    printf("\nOrder added successfully!\n");
}

void deq()
{
    int i, found = 0;
    struct Order o;

    if (f == -1)
    {
        printf("\nNo orders in the queue.\n");
        return;
    }

    o = q[f];

    if (f == r)
        f = r = -1;
    else
        f++;

    printf("\nProcessing Order %d (Product ID: %d, Quantity: %d)\n",
           o.orderId, o.productId, o.quantity);
```

```
    for (i = 0; i < productCount; i++)
    {
        if (products[i].productId == o.productId)
        {
            found = 1;

            if (products[i].quantity >= o.quantity)
            {
                products[i].quantity -= o.quantity;
                printf("Order processed successfully!\n");
                printf("Remaining Stock: %d\n", products[i].quantity);

                if (products[i].quantity <= products[i].reorderLevel)
                    printf("Low Stock Alert!\n");
            }
            else
            {
                printf("Not enough stock for this order\n");
            }
            break;
        }
    }

    if (!found)
        printf("Product not found!\n");
}

void displayQ()
{
    int i;

    if (f == -1)
    {
        printf("\nNo orders in the queue.\n");
        return;
    }

    printf("\n%-10s %-12s %s\n", "Order ID", "Product ID", "Quantity");
    printf("-----------------------------------------\n");
    for (i = f; i <= r; i++)
    {
        printf("%-10d %-12d %d\n",
               q[i].orderId,
               q[i].productId,
               q[i].quantity);
    }
}

void lowStockAlert()
{
    int found = 0;

    for (int i = 0; i < productCount; i++)
    {
        if (products[i].quantity <=
            products[i].reorderLevel)
        {
            printf("\nLow Stock Alert!\n");
            printf("Product ID: %d\n",
                   products[i].productId);
            printf("Product Name: %s\n",
                   products[i].productName);
            printf("Current Stock: %d\n",
                   products[i].quantity);
            printf("Reorder Level: %d\n",
                   products[i].reorderLevel);
            found = 1;
        }
    }

    if (!found)
        printf("\nNo products are currently low in stock.\n");
}

int main()
```

```
{
    int choice;

    while (1)
    {
        printf("\n================================================\n");
        printf("   SMART INVENTORY AND WAREHOUSE MANAGEMENT\n");
        printf("================================================\n");
        printf("1. Add Product\n");
        printf("2. Display Products\n");
        printf("3. Add Supplier\n");
        printf("4. Display Suppliers\n");
        printf("5. Update Stock\n");
        printf("6. Display Inventory\n");
        printf("7. Search Product\n");
        printf("8. Sort Products by Price\n");
        printf("9. Add Order\n");
        printf("10. Process Order\n");
        printf("11. Display Orders\n");
        printf("12. Low Stock Alert\n");
        printf("13. Exit\n");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1: addProduct(); break;
            case 2: displayProducts(); break;
            case 3: addSupplier(); break;
            case 4: displaySuppliers(); break;
            case 5: updateStock(); break;
            case 6: displayInventory(); break;
            case 7: search(); break;
            case 8: sort(); break;
            case 9: enq(); break;
            case 10: deq(); break;
            case 11: displayQ(); break;
            case 12: lowStockAlert(); break;
            case 13:
                printf("\nExiting program...\n");
                return 0;
            default:
                printf("\nInvalid choice!\n");
        }
    }

    return 0;
}
```

GitHub Link

https://github.com/kadiyalasaranya25cse-create

Plagiarism Detector .net

Plagiarism Report



Unique	100%
Exact Match	0%
Partial Match	0%

Primary Sources

Excluded URL (s)

01 None